Self-Attention Module — Re-Structured by DIPOD

Kevin T.N.
Independent Researcher
jkdkr2439@gmail.com

February 2026

1 Definition

This document specifies a production-ready Self-Attention module structured by **DIPOD**:

$$\textbf{I (Input)} \rightarrow \textbf{P (Process)} \rightarrow \textbf{O (Output)} \rightarrow \textbf{D (Data)} \rightarrow \textbf{Di (Display)}$$

- **I**: input contracts, validation, mask building, capability checks.
- **P**: compute path (projections, kernels, KV cache, strategy selection).
- **O**: packaging and return policy (no accidental heavy artifacts).
- **D**: telemetry/metrics/logging/diagnostics (scalar-only, sampled, non-fatal).
- **Di**: presentation/renderers (plots/reports) *read-only* from **O/D artifacts**; must not call compute.

2 Fractal DIPOD

Every scope is a DIPOD. The whole module is a DIPOD; submodules may also be DIPODs.

2.1 Directory Layout

```
self_attention_dipod/
  I/
    __init__.py
    input_contracts.py
    validation.py
    masks.py
    torch_compat.py
    capabilities.py

  P/
    __init__.py
    projections_mha.py
    projections_gqa.py
    repeat_kv.py
    mask_adapter.py
    kernels.py
    attention_core.py
    strategies.py
    kv_cache_allocator.py
    kv_cache.py
```

```
    distributed_attention.py
    kv_sharing.py

  O/
    __init__.py
    outputs.py
    packaging.py
    debug_payloads.py

  D/
    __init__.py
    metrics.py
    logging.py
    async_sink.py
    diagnostics.py

  Di/
    __init__.py
    render_api.py
    heatmap_renderer.py
    report_renderer.py

  tests/
    test_masks.py
    test_no_all_blocked.py
    test_equiv_reference_vs_sdpa.py
    test_amp_safe.py
    test_large_T_no_attn.py
    test_decode_cache_consistency.py
    test_gqa_shapes.py
    test_display_import_boundaries.py
    test_display_no_tensor_serialization.py

  self_attention_dipod.py
  README.md
```

2.2 Layer Interaction Rules (Strict)

- **I** may call **P/O/D/Di**.
- **P** may call **O/D** (must not call **Di**).
- **O** may call **D/Di**.
- **D** may call **Di** (but must not call **P**).
- **Di** may call **O/D** only (must not import/call **I/P**).

3 Hard Contracts (Correctness & Safety)

3.1 C1: Mask Semantics

All internal masks use **True=keep/valid**, **False=block/invalid**.

3.2 C2: No All-Blocked Keys

Padding mask must not result in all keys blocked for any batch item. Fail early.

3.3 C3: No Accidental $O(T^2)$ Materialization

- In production mode (`return_attn=False`), never allocate/return `[B,h,T,T]` weights.
- **D** must not force attention matrix creation; only compute matrix-based metrics if weights already exist.
- **Di** only renders from pre-produced small artifacts (CPU, downsampled, or summaries).

3.4 C4: Backend Fallback is One-Way

Fallback chain: `flash` → `sdpa` → `reference`. No retry loops.

3.5 C5: AMP Safety Policy

If `precision_policy=amp_safe`, compute scores and softmax in FP32 and cast back.

3.6 C6: Display Isolation

Di must not:

- import from **I** or **P**,
- call forward/compute,
- serialize GPU tensors.

4 I — Input Layer

4.1 I/input_contracts.py

```python
from __future__ import annotations
from dataclasses import dataclass
from typing import Literal, Optional

KernelName = Literal["flash", "sdpa", "reference"]
PrecisionPolicy = Literal["native", "amp_safe"]
AttentionType = Literal["mha", "gqa"]

@dataclass(frozen=True)
class SelfAttentionConfig:
    d_model: int
    num_heads: int
    dropout_p: float = 0.0

    attention_type: AttentionType = "mha"
    kv_num_heads: Optional[int] = None  # required if attention_type="gqa"

    kernel_preference: KernelName = "sdpa"
    precision_policy: PrecisionPolicy = "native"

    enable_telemetry: bool = True
    telemetry_sample_rate: float = 0.05

    enable_kv_cache: bool = True

    # Debug artifact policy
    enable_debug_payloads: bool = False
```

```
    debug_max_T_for_matrix: int = 512
```

4.2 I/validation.py

```
from __future__ import annotations
import torch
from .input_contracts import SelfAttentionConfig

class InputValidationError(ValueError):
    pass

def validate_inputs(x: torch.Tensor,
                    padding_mask: torch.Tensor | None,
                    causal: bool,
                    cfg: SelfAttentionConfig) -> None:
    if x.dim() != 3:
        raise InputValidationError(f"x must be [B,T,d_model], got {tuple(x.shape)}")
    B, T, d_model = x.shape

    if d_model != cfg.d_model:
        raise InputValidationError("d_model mismatch between x and cfg")
    if cfg.d_model % cfg.num_heads != 0:
        raise InputValidationError("d_model must be divisible by num_heads")

    if cfg.attention_type == "gqa":
        if cfg.kv_num_heads is None:
            raise InputValidationError("kv_num_heads must be set for gqa")
        if cfg.kv_num_heads <= 0 or cfg.kv_num_heads > cfg.num_heads:
            raise InputValidationError("kv_num_heads must be in (0, num_heads]")

    if padding_mask is not None:
        if padding_mask.shape != (B, T):
            raise InputValidationError("padding_mask must be [B,T]")
        if padding_mask.dtype != torch.bool:
            raise InputValidationError("padding_mask must be bool (True=valid)")

    if T <= 0:
        raise InputValidationError("T must be > 0")
```

4.3 I/masks.py (Padding-only keep-mask)

```
from __future__ import annotations
import torch

def compose_padding_keep_mask(padding_mask: torch.Tensor | None, B: int, T: int):
    # padding only; causal handled by backend (is_causal=True)
    if padding_mask is None:
        return None
    return padding_mask.view(B, 1, 1, T)  # [B,1,1,T], True=keep

```

```python
def ensure_not_all_blocked(padding_keep: torch.Tensor | None) -> None:
    if padding_keep is None:
        return
    # If all keys are blocked for any batch item -> invalid
    any_key = padding_keep.any(dim=-1)  # [B,1,1]
    if not any_key.all().item():
        raise ValueError("All keys blocked by padding mask for at least one batch item.")
```

4.4 I/torch_compat.py (Policy stub)

```python
from __future__ import annotations
import torch

def torch_version_tuple() -> tuple[int,int,int]:
    v = torch.__version__.split("+")[0]
    p = v.split(".")
    return (int(p[0]), int(p[1]), int(p[2]) if len(p) > 2 else 0)
```

4.5 I/capabilities.py

```python
from __future__ import annotations
import torch

class KernelCapabilities:
    def __init__(self):
        self._checked = False
        self.has_sdpa = False
        self.has_flash = False

    def check(self):
        if self._checked:
            return
        self._checked = True
        self.has_sdpa = hasattr(torch.nn.functional, "scaled_dot_product_attention")
        self.has_flash = False  # set True after integrating flash backend

CAPS = KernelCapabilities()
```

5 P — Process Layer

5.1 P/projections_mha.py

```python
from __future__ import annotations
import torch
import torch.nn as nn
from ..I.input_contracts import SelfAttentionConfig

class ProjectionsMHA(nn.Module):
    def __init__(self, cfg: SelfAttentionConfig):
        super().__init__()
        d = cfg.d_model
```

```
        self.wq = nn.Linear(d, d, bias=False)
        self.wk = nn.Linear(d, d, bias=False)
        self.wv = nn.Linear(d, d, bias=False)
        self.wo = nn.Linear(d, d, bias=False)

    def forward(self, x: torch.Tensor):
        return self.wq(x), self.wk(x), self.wv(x)
```

5.2 P/projections_gqa.py (Fixed head-dimension alignment)

```
from __future__ import annotations
import torch
import torch.nn as nn
from ..I.input_contracts import SelfAttentionConfig

class ProjectionsGQA(nn.Module):
    """
    Head-dimension contract:
      d_head = d_model // num_heads (query heads)
      K/V head dim MUST equal d_head.
      K/V projection output dim = kv_num_heads * d_head.
    """
    def __init__(self, cfg: SelfAttentionConfig):
        super().__init__()
        assert cfg.kv_num_heads is not None
        d_model = cfg.d_model
        Hq = cfg.num_heads
        Hkv = cfg.kv_num_heads
        d_head = d_model // Hq
        kv_dim = Hkv * d_head

        self.wq = nn.Linear(d_model, d_model, bias=False)  # Hq*d_head
        self.wk = nn.Linear(d_model, kv_dim, bias=False)   # Hkv*d_head
        self.wv = nn.Linear(d_model, kv_dim, bias=False)   # Hkv*d_head
        self.wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x: torch.Tensor):
        return self.wq(x), self.wk(x), self.wv(x)
```

5.3 P/repeat_kv.py

```
from __future__ import annotations
import torch

def repeat_kv(kv: torch.Tensor, num_q_heads: int) -> torch.Tensor:
    B, Hkv, T, d = kv.shape
    if Hkv == num_q_heads:
        return kv
    if num_q_heads % Hkv != 0:
        raise ValueError("num_q_heads must be multiple of kv_num_heads")
    r = num_q_heads // Hkv
    return kv.repeat_interleave(r, dim=1)
```

5.4 P/mask_adapter.py (SDPA-safe additive mask by default)

```python
from __future__ import annotations
import torch

def to_additive_mask(keep_mask: torch.Tensor) -> torch.Tensor:
    # keep=True -> 0, block=False -> -inf
    out = torch.zeros_like(keep_mask, dtype=torch.float32)
    return out.masked_fill(~keep_mask, float("-inf"))

def adapt_padding_for_sdpa(padding_keep: torch.Tensor | None) -> torch.Tensor | None:
    if padding_keep is None:
        return None
    # Safe default: additive mask. (Bool semantics may vary by build.)
    return to_additive_mask(padding_keep)
```

5.5 P/kernels.py

```python
from __future__ import annotations
import math
import torch
import torch.nn.functional as F
from .mask_adapter import adapt_padding_for_sdpa

def reference_attention(q, k, v, *,
                        padding_keep=None,
                        dropout_p=0.0,
                        training=False,
                        amp_safe=False,
                        is_causal=False):
    B, H, Tq, d = q.shape
    Tk = k.size(2)

    if is_causal:
        causal = torch.tril(torch.ones((Tq, Tk), dtype=torch.bool, device=q.device))
        causal = causal.view(1, 1, Tq, Tk)
        padding_keep = causal if padding_keep is None else (padding_keep & causal)

    if amp_safe:
        scores = (q.float() @ k.float().transpose(-2, -1)) / math.sqrt(d)
        if padding_keep is not None:
            scores = scores.masked_fill(~padding_keep, float("-inf"))
        attn = torch.softmax(scores, dim=-1)
        if dropout_p > 0:
            attn = torch.dropout(attn, p=dropout_p, train=training)
        out = attn.to(v.dtype) @ v
        return out, attn

    scores = (q @ k.transpose(-2, -1)) / math.sqrt(d)
    if padding_keep is not None:
        scores = scores.masked_fill(~padding_keep, float("-inf"))
    attn = torch.softmax(scores, dim=-1)
    if dropout_p > 0:
        attn = torch.dropout(attn, p=dropout_p, train=training)
```

```
    out = attn @ v
    return out, attn

def sdpa_attention(q, k, v, *,
                   padding_keep=None,
                   dropout_p=0.0,
                   training=False,
                   is_causal=False):
    attn_mask = adapt_padding_for_sdpa(padding_keep)
    out = F.scaled_dot_product_attention(
        q, k, v,
        attn_mask=attn_mask,
        dropout_p=dropout_p if training else 0.0,
        is_causal=is_causal
    )
    return out, None

def flash_attention(q, k, v, *,
                    padding_keep=None,
                    dropout_p=0.0,
                    training=False,
                    is_causal=False):
    # Optional integration point for a FlashAttention library.
    raise NotImplementedError("FlashAttention backend not integrated.")
```

5.6 P/attention_core.py

```
from __future__ import annotations
from .kernels import reference_attention, sdpa_attention, flash_attention

class AttentionCore:
    def __init__(self, kernel: str):
        self.kernel = kernel

    def __call__(self, q, k, v, *,
                 padding_keep,
                 dropout_p,
                 training,
                 is_causal,
                 amp_safe):
        if self.kernel == "flash":
            return flash_attention(
                q, k, v,
                padding_keep=padding_keep,
                dropout_p=dropout_p,
                training=training,
                is_causal=is_causal
            )
        if self.kernel == "sdpa":
            return sdpa_attention(
                q, k, v,
                padding_keep=padding_keep,
                dropout_p=dropout_p,
                training=training,
                is_causal=is_causal
            )
        return reference_attention(
```

```
            q, k, v,
            padding_keep=padding_keep,
            dropout_p=dropout_p,
            training=training,
            is_causal=is_causal,
            amp_safe=amp_safe
        )
```

5.7 P/strategies.py

```python
from __future__ import annotations
from ..I.capabilities import CAPS

FALLBACK = {"flash": "sdpa", "sdpa": "reference", "reference": None}

class KernelSelector:
    def __init__(self):
        CAPS.check()

    def select(self, preferred: str) -> str:
        k = preferred
        while k is not None:
            if k == "flash" and CAPS.has_flash:
                return "flash"
            if k == "sdpa" and CAPS.has_sdpa:
                return "sdpa"
            if k == "reference":
                return "reference"
            k = FALLBACK.get(k)
        return "reference"

    def fallback(self, current: str):
        return FALLBACK.get(current)
```

5.8 P/kv_cache_allocator.py (Production allocator; avoids per-step cat)

```python
from __future__ import annotations
from dataclasses import dataclass
import torch

@dataclass
class KVStorage:
    k: torch.Tensor  # [B,H,capacity,d_head]
    v: torch.Tensor  # [B,H,capacity,d_head]
    length: int      # used length

class KVAllocator:
    def __init__(self, capacity: int):
        self.capacity = capacity

    def init(self, B: int, H: int, d_head: int, device, dtype) -> KVStorage:
        k = torch.empty((B, H, self.capacity, d_head), device=device, dtype=dtype)
        v = torch.empty((B, H, self.capacity, d_head), device=device, dtype=dtype)
```

```
        return KVStorage(k=k, v=v, length=0)

    def append(self, st: KVStorage, k_new: torch.Tensor, v_new: torch.Tensor) -> KVStorage:
        if st.length >= self.capacity:
            raise RuntimeError("KV cache capacity exceeded; implement growth strategy.")
        st.k[:, :, st.length:st.length+1, :] = k_new
        st.v[:, :, st.length:st.length+1, :] = v_new
        st.length += 1
        return st
```

5.9 P/kv_cache.py

```
from __future__ import annotations
from dataclasses import dataclass
import torch
from .kv_cache_allocator import KVAllocator, KVStorage

@dataclass
class KVCache:
    storage: KVStorage

    @property
    def length(self) -> int:
        return self.storage.length

    def keys(self) -> torch.Tensor:
        return self.storage.k[:, :, :self.storage.length, :]

    def values(self) -> torch.Tensor:
        return self.storage.v[:, :, :self.storage.length, :]

def init_cache(alloc: KVAllocator, B: int, H: int, d_head: int, device, dtype) -> KVCache:
    return KVCache(storage=alloc.init(B, H, d_head, device, dtype))

def append_kv(alloc: KVAllocator, cache: KVCache, k_new: torch.Tensor, v_new: torch.Tensor) -> KVCache:
    alloc.append(cache.storage, k_new, v_new)
    return cache
```

6 O — Output Layer

6.1 O/outputs.py

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Any, Optional
import torch

@dataclass
class SelfAttentionOutput:
    y: torch.Tensor
```

```
    attn_weights: Optional[torch.Tensor]   # only if explicitly requested and safe
    debug_payload: Optional[dict[str, Any]]  # small, CPU-friendly artifacts
    telemetry: Optional[dict[str, Any]]    # scalar-only
```

6.2 O/debug_payloads.py (Small artifacts only)

```
from __future__ import annotations
import torch

def make_debug_payload(attn: torch.Tensor | None,
                       max_T: int,
                       topk: int = 16) -> dict | None:
    """
    Produce small artifacts for Display without returning full matrices at large T.
    - If attn is None: return None
    - If T > max_T: return top-k summaries per query (downsampled)
    """
    if attn is None:
        return None
    B, H, T, _ = attn.shape
    if T <= max_T:
        # Safe: convert a small slice to CPU for visualization
        return {
            "attn_cpu": attn.detach().float().cpu()
        }
    # Summarize: top-k keys per query for a subset of heads (head0 only here)
    head0 = attn[:, 0, :, :]  # [B,T,T]
    vals, idx = torch.topk(head0, k=min(topk, T), dim=-1)
    return {
        "topk_idx_cpu": idx.detach().cpu(),
        "topk_val_cpu": vals.detach().float().cpu()
    }
```

6.3 O/packaging.py

```
from __future__ import annotations
import torch
from .outputs import SelfAttentionOutput

def _to_scalar(v):
    if isinstance(v, torch.Tensor):
        return v.detach().float().mean().item()
    return v

def pack_output(y: torch.Tensor,
                attn_weights: torch.Tensor | None,
                debug_payload: dict | None,
                telemetry: dict | None) -> SelfAttentionOutput:
    if telemetry is not None:
        telemetry = {k: _to_scalar(v) for k, v in telemetry.items()}
    return SelfAttentionOutput(
```

```
        y=y,
        attn_weights=attn_weights,
        debug_payload=debug_payload,
        telemetry=telemetry
    )
```

7 D — Data Layer (Telemetry, Metrics, Diagnostics)

7.1 D/metrics.py

```
from __future__ import annotations
import torch

def attention_entropy(attn: torch.Tensor, eps: float = 1e-9) -> torch.Tensor:
    p = attn.clamp_min(eps)
    return -(p * p.log()).sum(dim=-1).mean()

def pad_leak_ratio(attn: torch.Tensor, padding_mask: torch.Tensor) -> torch.Tensor:
    invalid = (~padding_mask).view(padding_mask.size(0), 1, 1, padding_mask.size(1))
    return (attn * invalid).sum(dim=-1).mean()

def head_similarity(attn: torch.Tensor) -> torch.Tensor:
    B, H, T, _ = attn.shape
    a = attn.view(B, H, -1)
    a = a / (a.norm(dim=-1, keepdim=True) + 1e-9)
    sim = torch.matmul(a, a.transpose(1, 2))
    off = (sim.sum(dim=(1,2)) - sim.diagonal(dim1=1, dim2=2).sum(dim=1)) / (H*H - H)
    return off.mean()
```

7.2 D/logging.py

```
from __future__ import annotations

class TelemetrySink:
    def emit(self, event: dict) -> None:
        pass

DEFAULT_SINK = TelemetrySink()

def safe_emit(sink: TelemetrySink, event: dict, enabled: bool) -> None:
    if not enabled:
        return
    try:
        sink.emit(event)
    except Exception:
        return
```

7.3 D/async_sink.py

```python
from __future__ import annotations
import queue
import threading
from .logging import TelemetrySink

class AsyncTelemetrySink(TelemetrySink):
    def __init__(self, inner: TelemetrySink, maxsize: int = 1024):
        self.inner = inner
        self.q = queue.Queue(maxsize=maxsize)
        self.t = threading.Thread(target=self._worker, daemon=True)
        self.t.start()

    def _worker(self):
        while True:
            event = self.q.get()
            try:
                self.inner.emit(event)
            except Exception:
                pass
            finally:
                self.q.task_done()

    def emit(self, event: dict) -> None:
        try:
            self.q.put_nowait(event)
        except queue.Full:
            return
```

7.4 D/diagnostics.py (Sampling gate; no forced matrix creation)

```python
from __future__ import annotations
import random
import time
import torch
from .metrics import attention_entropy, head_similarity, pad_leak_ratio

def maybe_make_telemetry(attn: torch.Tensor | None,
                         padding_mask: torch.Tensor | None,
                         kernel: str,
                         B: int, T: int, d_model: int, num_heads: int,
                         start_time: float,
                         sample_rate: float) -> dict | None:
    if sample_rate <= 0.0:
        return None
    if random.random() >= sample_rate:
        return None

    t = {
        "kernel": kernel,
        "B": B,
        "T": T,
        "d_model": d_model,
        "num_heads": num_heads,
        "latency_ms": (time.time() - start_time) * 1000.0,
    }

```

```
    # Only compute matrix-based metrics if attn already exists (reference path).
    if attn is not None:
        t["attn_entropy"] = attention_entropy(attn).item()
        t["head_similarity"] = head_similarity(attn).item() if num_heads > 1 else 0.0
        if padding_mask is not None:
            t["pad_leak_ratio"] = pad_leak_ratio(attn, padding_mask).item()

    return t
```

8 Di — Display Layer (Renderers, Read-Only)

8.1 Di/render_api.py

```
from __future__ import annotations
from typing import Protocol, Any

class Renderer(Protocol):
    def render(self, payload: dict[str, Any]) -> Any:
        ...
```

8.2 Di/heatmap_renderer.py

```
from __future__ import annotations
from typing import Any
import numpy as np

class HeatmapRenderer:
    """
    Input payload must be CPU-friendly:
      - either 'attn_cpu' tensor converted to numpy by caller
      - or top-k indices/values.
    """
    def render(self, payload: dict) -> Any:
        if "attn_cpu" in payload:
            attn = payload["attn_cpu"]
            if hasattr(attn, "numpy"):
                attn = attn.numpy()
            # Return a numpy array for downstream plotting
            return np.asarray(attn)
        if "topk_idx_cpu" in payload and "topk_val_cpu" in payload:
            idx = payload["topk_idx_cpu"]
            val = payload["topk_val_cpu"]
            return {"topk_idx": idx.numpy(), "topk_val": val.numpy()}
        raise ValueError("Unsupported payload for HeatmapRenderer")
```

9 Main Assembly (self_attention_dipod.py)

```
from __future__ import annotations
import time
import torch
import torch.nn as nn
```

```python

from .I.input_contracts import SelfAttentionConfig
from .I.validation import validate_inputs
from .I.masks import compose_padding_keep_mask, ensure_not_all_blocked

from .P.strategies import KernelSelector
from .P.attention_core import AttentionCore
from .P.projections_mha import ProjectionsMHA
from .P.projections_gqa import ProjectionsGQA
from .P.repeat_kv import repeat_kv
from .P.kv_cache_allocator import KVAllocator
from .P.kv_cache import KVCache, init_cache, append_kv

from .O.debug_payloads import make_debug_payload
from .O.packaging import pack_output

from .D.diagnostics import maybe_make_telemetry
from .D.logging import DEFAULT_SINK, safe_emit


class SelfAttentionDIPOD(nn.Module):
    def __init__(self, cfg: SelfAttentionConfig, telemetry_sink=DEFAULT_SINK):
        super().__init__()
        self.cfg = cfg
        self.telemetry_sink = telemetry_sink

        if cfg.attention_type == "gqa":
            self.proj = ProjectionsGQA(cfg)
        else:
            self.proj = ProjectionsMHA(cfg)

        self.kernel_selector = KernelSelector()

    def _split_heads(self, t: torch.Tensor, B: int, T: int, H: int, d_head: int) -> torch.Tensor:
        return t.view(B, T, H, d_head).transpose(1, 2)

    def forward(self, x: torch.Tensor,
                padding_mask: torch.Tensor | None = None,
                causal: bool = False,
                return_attn: bool = False):
        t0 = time.time()
        validate_inputs(x, padding_mask, causal, self.cfg)

        B, T, _ = x.shape
        Hq = self.cfg.num_heads
        d_head = self.cfg.d_model // Hq

        # I: padding keep-mask only (causal handled by backend)
        padding_keep = compose_padding_keep_mask(padding_mask, B=B, T=T)
        ensure_not_all_blocked(padding_keep)

        # P: projections
        q_lin, k_lin, v_lin = self.proj(x)
        q = self._split_heads(q_lin, B, T, Hq, d_head)

        if self.cfg.attention_type == "gqa":
```

```
            Hkv = int(self.cfg.kv_num_heads)
            k = self._split_heads(k_lin, B, T, Hkv, d_head)
            v = self._split_heads(v_lin, B, T, Hkv, d_head)
            k = repeat_kv(k, Hq)
            v = repeat_kv(v, Hq)
        else:
            k = self._split_heads(k_lin, B, T, Hq, d_head)
            v = self._split_heads(v_lin, B, T, Hq, d_head)

        kernel = self.kernel_selector.select(self.cfg.kernel_preference)
        amp_safe = (self.cfg.precision_policy == "amp_safe")

        attn_weights = None
        while True:
            try:
                core = AttentionCore(kernel)
                out, attn = core(
                    q, k, v,
                    padding_keep=padding_keep,
                    dropout_p=self.cfg.dropout_p,
                    training=self.training,
                    is_causal=causal,
                    amp_safe=amp_safe
                )
                attn_weights = attn
                break
            except Exception:
                nxt = self.kernel_selector.fallback(kernel)
                if nxt is None:
                    raise
                kernel = nxt

        out = out.transpose(1, 2).contiguous().view(B, T, self.cfg.d_model)
        y = self.proj.wo(out)

        # D: telemetry (sampled; matrix metrics only if attn exists)
        telemetry = maybe_make_telemetry(
            attn=attn_weights,
            padding_mask=padding_mask,
            kernel=kernel,
            B=B, T=T, d_model=self.cfg.d_model, num_heads=Hq,
            start_time=t0,
            sample_rate=self.cfg.telemetry_sample_rate
        )
        if self.cfg.enable_telemetry and telemetry is not None:
            safe_emit(self.telemetry_sink, telemetry, enabled=True)

        # O: debug payload (small artifacts only)
        debug_payload = None
        if self.cfg.enable_debug_payloads:
            debug_payload = make_debug_payload(
                attn=attn_weights,
                max_T=self.cfg.debug_max_T_for_matrix
            )

        # O: return policy for attention weights
        if not return_attn:
            attn_weights = None
```

```
        else:
            # safety: only allow returning full matrix if small enough and exists
            if attn_weights is None or T > self.cfg.debug_max_T_for_matrix:
                attn_weights = None

        return pack_output(
            y=y,
            attn_weights=attn_weights,
            debug_payload=debug_payload,
            telemetry=telemetry
        )

    @torch.no_grad()
    def decode_step(self, x_t: torch.Tensor,
                    cache: KVCache | None,
                    alloc: KVAllocator,
                    padding_mask: torch.Tensor | None = None,
                    return_attn: bool = False):
        if not self.cfg.enable_kv_cache:
            raise RuntimeError("KV cache disabled in config")

        B, Tstep, _ = x_t.shape
        if Tstep != 1:
            raise ValueError("decode_step expects x_t [B,1,d_model]")

        Hq = self.cfg.num_heads
        d_head = self.cfg.d_model // Hq
        Hkv = int(self.cfg.kv_num_heads) if self.cfg.attention_type == "gqa" else Hq

        if cache is None:
            cache = init_cache(alloc, B=B, H=Hkv, d_head=d_head, device=x_t.device, dtype=x_t.dtype)

        q_lin, k_lin, v_lin = self.proj(x_t)
        q = self._split_heads(q_lin, B, 1, Hq, d_head)

        if self.cfg.attention_type == "gqa":
            k_new = self._split_heads(k_lin, B, 1, Hkv, d_head)
            v_new = self._split_heads(v_lin, B, 1, Hkv, d_head)
        else:
            k_new = self._split_heads(k_lin, B, 1, Hq, d_head)
            v_new = self._split_heads(v_lin, B, 1, Hq, d_head)

        cache = append_kv(alloc, cache, k_new, v_new)

        k = cache.keys()
        v = cache.values()
        if self.cfg.attention_type == "gqa":
            k = repeat_kv(k, Hq)
            v = repeat_kv(v, Hq)

        padding_keep = None
        if padding_mask is not None:
            if padding_mask.size(1) != cache.length:
                raise ValueError("padding_mask length must match cache length")
```

```python
            padding_keep = padding_mask.view(B, 1, 1, cache.length)
            ensure_not_all_blocked(padding_keep)

        kernel = self.kernel_selector.select(self.cfg.kernel_preference)
        amp_safe = (self.cfg.precision_policy == "amp_safe")

        attn_weights = None
        while True:
            try:
                core = AttentionCore(kernel)
                out, attn = core(
                    q, k, v,
                    padding_keep=padding_keep,
                    dropout_p=0.0,
                    training=False,
                    is_causal=False,
                    amp_safe=amp_safe
                )
                attn_weights = attn
                break
            except Exception:
                nxt = self.kernel_selector.fallback(kernel)
                if nxt is None:
                    raise
                kernel = nxt

        out = out.transpose(1, 2).contiguous().view(B, 1, self.cfg.d_model)
        y = self.proj.wo(out)

        # decode returns no debug payload by default
        if not return_attn:
            attn_weights = None

        return pack_output(y=y, attn_weights=attn_weights,
    debug_payload=None, telemetry=None), cache
```

10 Tests (Required)

10.1 tests/test_display_import_boundaries.py

```python
import importlib
import pkgutil

def test_display_does_not_import_process_or_input():
    # crude but effective: ensure Di package can be imported without
    importing I/P.
    di = importlib.import_module("self_attention_dipod.Di")
    assert di is not None
```

10.2 tests/test_gqa_shapes.py

```python
import torch
from self_attention_dipod.I.input_contracts import SelfAttentionConfig
from self_attention_dipod.self_attention_dipod import SelfAttentionDIPOD

```

```python
def test_gqa_head_dim_alignment():
    cfg = SelfAttentionConfig(d_model=512, num_heads=8, attention_type="gqa", kv_num_heads=2)
    sa = SelfAttentionDIPOD(cfg)
    x = torch.randn(2, 16, 512)
    out = sa(x, causal=True, return_attn=False)
    assert out.y.shape == (2, 16, 512)
```

11 Notes

- **Di** is intentionally thin and read-only to preserve scale.
- Debug visuals must use **O.debug_payload** artifacts; do not request full matrices at large `T`.
- Replace the flash backend stub only behind `P/kernels.flash_attention()`.